# TP3 - Entrega continua y resiliencia

Materia: Metodología de Sistemas II
Unidad 4: Repositorios, colaboración y entrega continua
Alumno: Nicolás Viruel

## Parte 1: Integración continua (CI)

### 1. Filosofía Fail-Fast

Fallar temprano (lint barato antes que suite E2E cara) da feedback en minutos, evita apilar commits sobre base rota y ahorra runners y tiempo de equipo. Aumenta agilidad porque el autor corrige en contexto fresco.

### 2. Stages del pipeline

Etapa — Qué valida — Riesgo mitigado
Lint & static analysis — Estilo, smells, posibles bugs estáticos — Deuda, bugs evitables, inconsistencias
Unit tests — Lógica aislada, regresiones — Comportamiento incorrecto en módulos
Security scan — Dependencias vulnerables, secretos, SAST básico — Exposición y cumplimiento

### 3. Pipeline as Code (YAML en repo)

El pipeline versiona junto al código: mismos criterios en cada rama/PR, revisable en PR, reproducible y auditable. Cambios en build/tests viajan con el cambio que los requiere.

## Parte 2: SemVer

### 4. Versión actual v2.4.1

Escenario — Nueva versión — Regla
A - fix impuestos sin cambiar uso — v2.4.2 — PATCH
B - export PDF compatible — v2.5.0 — MINOR
C - breaking auth y eliminación de API — v3.0.0 — MAJOR

### 5. Tags y despliegue

Un tag anotado en Git (p. ej. v2.5.0) es evento estable. El pipeline CD se dispara al push del tag: construye artefacto, lo promueve a prod/staging según reglas y asocia release notes al identificador SemVer.

## Parte 3: CD y entornos

### 6. Delivery vs Deployment

• Continuous Delivery: artefacto listo para prod; despliegue final puede requerir aprobación humana (compliance, ventanas).
• Continuous Deployment: cada cambio aprobado por CI llega solo a prod.

Preferir Delivery en dominios regulados (salud, banca) donde un humano autoriza el paso a producción pese a CI verde.

### 7. Dev → Staging → Prod

Dev: integración y prueba rápida. Staging: réplica cercana a prod (datos anonimizados, mismos servicios); validación E2E, UAT y performance antes del corte. Prod: usuarios reales. Staging es último filtro antes de impacto masivo.

### 8. Artefacto inmutable

Es el mismo binario/imagen (hash fijo) promovido entre entornos; solo cambian configuración y secretos. No recompilar en prod evita “funcionaba en staging” por diferencias de build y permite rollback confiable al artefacto anterior.

## Parte 4: Resiliencia

### 9. Rollback

• Rollback: volver la infraestructura/tráfico a la versión anterior (p. ej. redeploy imagen v2.4.2 o switch Blue-Green).
• Tags SemVer identifican releases exactos; el pipeline ya tiene el artefacto etiquetado → rollback = redeploy del tag previo, casi instantáneo.

### 10. Blue-Green vs Canary

Blue-Green — Canary
Tráfico — 100% salta de entorno viejo (blue) a nuevo (green) o viceversa — Porcentaje gradual al nuevo (5% → 50% → 100%)
Prueba con usuarios reales limitados — No (corte total) — Sí - grupo reducido primero

Canary permite validar en prod con riesgo acotado antes del lanzamiento total.